Object-Oriented Programming Using Java

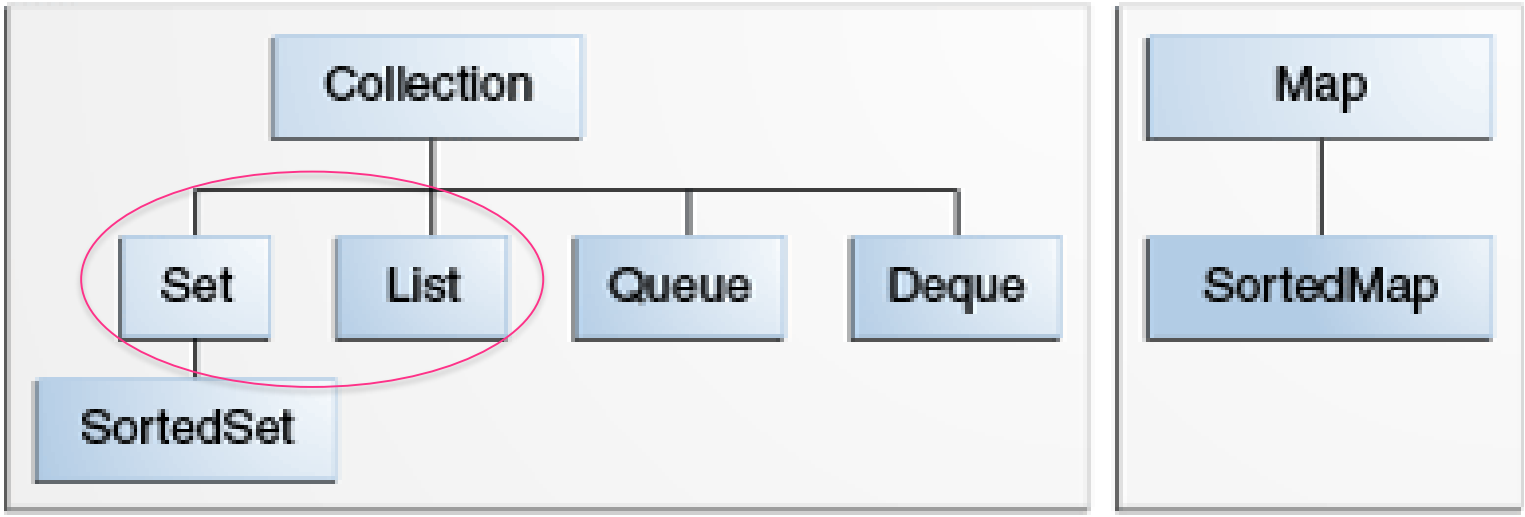
Collections

Contents

- TreeSet
- ArrayList
- Sorting Algorithms

References:
https://docs.oracle.com/javase/tutorial/collections/index.html

The core collection interfaces



Implementations					
Set	HashSet		TreeSet		LinkedHashSet
List		ArrayList		LinkedList	
Queue					
Deque		ArrayDeque		LinkedList	
Map	HashMap		TreeMap		LinkedHashMap

Implementations

- Set — **không chứa phần tử trùng nhau**.
  - TreeSet: implementing the interface Set
    - **sorted** according to the *natural ordering* of elements
- List — **có thể chứa các phần tử trùng nhau**.
  - ArrayList: implementing the interface List

Vd1: ArrayList và TreeSet

```
public static void main(String[] args) {
    TreeSet ts = new TreeSet();
    ArrayList list = new ArrayList();

    // Add elements to ArrayList and TreeSet
    for (int i = 10; i > 0; i--) {
        ts.add(i);
        list.add(i);
    }
    //Add duplicate elements
    ts.add(1);
    list.add(1);

    //output ArrayList
    System.out.print("ArrayList: ");
    System.out.println(list);
    //output TreeSet
    System.out.print("TreeSet: ");
    System.out.println(ts);
}
```

ArrayList: [10, 9, 8, 7, 6, 5, 4, 3, 2, **1, 1**]

TreeSet: [**1**, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Duyệt các phần tử của List

```
//using for
for (int i = 0; i < list.size(); i++) {
    System.out.println(list.get(i));
}

//using foreach
for (Object obj:list){
    System.out.println(obj);
}

//using Iterator
Iterator iter = list.iterator();
while (iter.hasNext()){
    System.out.println(iter.next());
}

//using for and Iterator
iter = list.iterator();
for (int i = 0; i < list.size(); i++) {
    System.out.println(iter.next());
}
```

Vd2: phần tử của TreeSet là Object

```
public static void main(String[] args) {
    TreeSet ts = new TreeSet();
    //Add students to TreeSet
    ts.add(new Student(1, "Name 01"));
    ts.add(new Student(7, "Name 07"));
    ts.add(new Student(5, "Name 05"));
    ts.add(new Student(10, "Name 10"));
    ts.add(new Student(2, "Name 02"));
    //Add duplicate elements
    ts.add(new Student(2, "Name 02"));

    //output TreeSet
    System.out.println(ts);
}
```

Sorted according to id or name???

Run-time error:
java.lang.ClassCastException

Khắc phục lỗi của Vd2

```
public interface Comparable<T> {
    public int compareTo(T o);
}
```

```
class Student implements Comparable<Student>{
    @Override
    public int compareTo(Student obj) {
        //Sort by id ascending
        if (id > obj.id){
            return 1; // positive integer
        }
        else if (id == obj.id){
            return 0;
        }
        else{
            return -1; // negative integer
        }
    }
```

Implementing *compareTo* method

```
class Student implements Comparable<Student>{
    int id;
    String name;
    public Student(int id, String name) {
        this.id = id;
        this.name = name;
    }
    @Override
    public int compareTo(Student obj) {
        if (id > obj.id){
            return 1; // positive integer
        }
        else if (id == obj.id){
            return 0;
        }
        else{
            return -1; // negative integer
        }
    }
    @Override
    public String toString(){
        return id+"-"+name;
    }
}
```

Vd2 output:

```
run:
[1-Name 01, 2-Name 02, 5-Name 05, 7-Name 07, 10-Name 10]
```

Sort List

```
public static void main(String[] args) {
    List list = new ArrayList();
    //Add students to ArrayList
    list.add(new Student(1, "Name 01"));
    list.add(new Student(7, "Name 07"));
    list.add(new Student(5, "Name 05"));
    list.add(new Student(10, "Name 10"));
    list.add(new Student(2, "Name 021"));
    //Add duplicate elements
    list.add(new Student(2, "Name 022"));

    //sort list
    Collections.sort(list);

    System.out.println(list);
}
```

Output: including duplicate elements

```
run:
[1-Name 01, 2-Name 021, 2-Name 022, 5-Name 05, 7-Name 07, 10-Name 10]
```

Vd3: sắp xếp theo id tăng dần và name tăng dần

```
public int compareTo(Student obj) {
    //Sort by id ascending
    if (id > obj.id){
        return 1; // positive integer
    }
    else if (id == obj.id){
        if (name.compareTo(obj.name) > 0) return 1;
        else if (name.compareTo(obj.name) == 0) return 0;
        else return -1;
    }
    else{
        return -1; // negative integer
    }
}
```

```
return name.compareTo(obj.name);
```

Vd3 - compareTo

```
public int compareTo(Student obj) {
    //Sort by id ascending
    if (id > obj.id){
        return 1; // positive integer
    }
    else if (id == obj.id){
        return name.compareTo(obj.name);
    }
    else{
        return -1; // negative integer
    }
}
```

Sử dụng Comparator Object

```
Comparator<Student> compObj = new Comparator<Student>() {
    @Override
    public int compare(Student o1, Student o2) {
        if (o1.id > o2.id){
            return 1;
        }
        else if (o1.id == o2.id){
            return 0;
        }
        else{
            return -1;
        }
    }
};
```

```
//sort list
Collections.sort(list, compObj);
```

Vd4: sắp xếp theo id tăng dần và name giảm dần

```
Comparator<Student> compObj = new Comparator<Student>() {
    @Override
    public int compare(Student o1, Student o2) {
        if (o1.id > o2.id){
            return 1;
        }
        else if (o1.id == o2.id){
            return -o1.name.compareTo(o2.name);
        }
        else{
            return -1;
        }
    }
};
```

Tóm tắt

- **TreeSet**: tự động sắp xếp, không nhận phần tử trùng
- **ArrayList**: có thể chứa các phần tử trùng nhau, gọi hàm Collections.sort(list) để sắp xếp
- Hiện thực hàm **compareTo()** để qui định kiểu sắp xếp
- Sử dụng **Comparator** Object để lựa chọn nhiều kiểu sắp xếp

Class java.util.ArrayList<E>

Return Type	Method and Description
boolean	**add(E e)** Appends the specified element to the end of this list.
void	**add(int index, E element)** Inserts the specified element at the specified position in this list.
void	**clear()** Removes all of the elements from this list.
Object	**clone()** Returns a shallow copy of this ArrayList instance.
boolean	**contains(Object o)** Returns true if this list contains the specified element.
E	**get(int index)** Returns the element at the specified position in this list.

Return Type	Method and Description
int	**indexOf(Object o)** Returns the index of the first occurrence of the specified element in this list, or -1 if this list does not contain the element.
boolean	**isEmpty()** Returns true if this list contains no elements.
iterator<E>	**iterator()** Returns an iterator over the elements in this list in proper sequence.
E	**remove(int index)** Removes the element at the specified position in this list.
boolean	**remove(Object o)** Removes the first occurrence of the specified element from this list, if it is present.
int	**size()** Returns the number of elements in this list.
void	**sort(Comparator<? super E> c)** Sorts this list according to the order induced by the specified Comparator.